

Name: P Neeharika

Roll No: 2520090233

Practical session 8

TASK 1

Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
GNU nano 7.2                                memory.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *a, *b;

    // malloc
    a = (int *)malloc(3 * sizeof(int));

    if (a == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    a[0] = 10;
    a[1] = 20;
    a[2] = 30;

    printf("Using malloc: %d %d %d\n", a[0], a[1], a[2]);

    // calloc
    b = (int *)calloc(3, sizeof(int));

    if (b == NULL) {
        printf("Memory allocation failed\n");
        free(a);
        return 1;
    }

    printf("Using calloc: %d %d %d\n", b[0], b[1], b[2]);

    // realloc
    a = (int *)realloc(a, 5 * sizeof(int));

    if (a == NULL) {
        printf("Reallocation failed\n");
        free(b);
        return 1;
    }

    a[3] = 40;
    a[4] = 50;

    printf("After realloc: %d %d %d %d %d\n",
        a[0], a[1], a[2], a[3], a[4]);

    // free
    free(a);
    free(b);

    printf("Memory freed successfully.\n");
}
```

```
root@HP:~# nano memory.c
root@HP:~# nano memory.c
root@HP:~# gcc memory.c -o memory
root@HP:~# ./memory
Using malloc: 10 20 30
Using calloc: 0 0 0
After realloc: 10 20 30 40 50
Memory freed successfully.
root@HP:~# |
```

```
valgrind-3.22.0
root@HP:~# valgrind --leak-check=full ./memory
==752== Memcheck, a memory error detector
==752== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al
.
==752== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright
info
==752== Command: ./memory
==752==
Using malloc: 10 20 30
Using calloc: 0 0 0
After realloc: 10 20 30 40 50
Memory freed successfully.
==752==
==752== HEAP SUMMARY:
==752==   in use at exit: 0 bytes in 0 blocks
==752==   total heap usage: 4 allocs, 4 frees, 1,068 bytes allocated
==752==
==752== All heap blocks were freed -- no leaks are possible
==752==
==752== For lists of detected and suppressed errors, rerun with: -s
==752== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@HP:~# |
```

```
root@HP:~# valgrind --leak-check=full ./memory
==760== Memcheck, a memory error detector
==760== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al
.
==760== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright
info
==760== Command: ./memory
==760==
Using malloc: 10 20 30
Using calloc: 0 0 0
After realloc: 10 20 30 40 50
Memory freed successfully.
==760==
==760== HEAP SUMMARY:
==760==   in use at exit: 12 bytes in 1 blocks
==760==   total heap usage: 4 allocs, 3 frees, 1,068 bytes allocated
==760==
==760== 12 bytes in 1 blocks are definitely lost in loss record 1 of 1
==760==   at 0x484D953: calloc (in /usr/libexec/valgrind/vgpreload_me
mcheck-amd64-linux.so)
==760==   by 0x109287: main (in /root/memory)
==760==
==760== LEAK SUMMARY:
==760==   definitely lost: 12 bytes in 1 blocks
==760==   indirectly lost: 0 bytes in 0 blocks
==760==   possibly lost: 0 bytes in 0 blocks
==760==   still reachable: 0 bytes in 0 blocks
==760==     suppressed: 0 bytes in 0 blocks
==760==
==760== For lists of detected and suppressed errors, rerun with: -s
==760== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
root@HP:~# |
```

TASK 2

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>

#define SIZE (10 * 1024 * 1024)

void show_memory(const char *message) {
    struct rusage usage;

    getrusage(RUSAGE_SELF, &usage);

    printf("%s: %ld KB\n", message, usage.ru_maxrss);
}

int main() {
    char *data;

    data = (char *)malloc(SIZE);

    if (data == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < SIZE; i++) {
        data[i] = 0;
    }

    show_memory("Parent memory before fork");

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        free(data);
    }
}
```

process.

```

        return 1;
    }

    if (pid == 0) {
        printf("\nChild process created.\n");

        show_memory("Child memory before modification");

        for (int i = 0; i < SIZE; i++) {
            data[i] = 1;
        }

        show_memory("Child memory after modification");

        printf("Child modified the data.\n");

        free(data);
    }
    else {
        wait(NULL);

        show_memory("Parent memory after child finishes");

        printf("Parent data is unchanged.\n");

        free(data);
    }

    return 0;
}

```

```

root@HP:~# ./cow
Parent memory before fork: 11364 KB

Child process created.
Child memory before modification: 10776 KB
Child memory after modification: 10904 KB
Child modified the data.
Parent memory after child finishes: 11752 KB
Parent data is unchanged.
root@HP:~# |

```

FINAL OBSERVATION

The programs successfully demonstrated dynamic memory allocation, memory leak detection using Valgrind, and Copy-on-Write behavior after `fork()`.